

SEDS 520 Microservice Project Report

1. Service Information

- **Microservice / Component name:** Bidding Service / Auction & Bidding Service
- **Team members:** Ufuk Cem Delice, Egehan Vardar, Mustafa Şengül
- **Main responsibility:** Manage auction sessions and basket bidding, maintain local buyer and basket projections, determine sold and unsold outcomes, support rebid rounds, and publish bidding-related events.
- **GitHub repository link:** <https://github.com/UfukCemDELICE/bidding-service>

The repository includes source code, a README file, and running instructions. It also includes package metadata, environment examples, database schema files, and Kafka integration code.

2. Service Description

This service manages the live auction phase of an online fish auction system. It builds local projections of buyers and catalog baskets from upstream Kafka events, creates auction sessions when a catalog is published, allows operators to start sessions and open baskets, accepts bids, closes baskets as sold or unsold, and manages rebid rounds.

Its main role in the overall system is to execute the bidding workflow between catalog publication and post-auction processing. Based on topic names, README documentation, and code, it interacts with:

- a buyer-related upstream service through `user.buyer.registered`
- a catalog-related upstream service through `catalog.basket.created` and `catalog.published`
- a downstream post-auction or fulfillment service through published auction and bid events

The relation to specific service names such as User Service, Catalog Service, and Post-Auction & Fulfillment is supported by the README and is partly inferred from topic naming.

3. Requirements / User Stories

ID	Requirement / User Story
R-01	The service shall consume buyer registration events and maintain a local buyer projection for bid validation.

- R-02 The service shall consume basket creation events and maintain a local basket projection including base price and descriptive data.
- R-03 The service shall consume catalog publication events and create local auction sessions and listed baskets.
- R-04 The service shall allow an operator to start an auction session only when the session status is **READY**.
- R-05 The service shall allow an operator to open a basket only when its status is **LISTED** or **REBID_READY** and its pricing details are available.
- R-06 The service shall accept a bid only when the session is **LIVE**, the basket is **OPEN**, the buyer exists locally, and the bid amount exceeds the current highest bid and meets the base price.
- R-07 The service shall record valid bids and update the basket's highest bid value.
- R-08 The service shall close an open basket as **SOLD** when at least one valid bid exists.
- R-09 The service shall close an open basket as unsold, queue it for rebid on the first no-bid closure, and mark it **FINAL_UNSOLD** if it closes again without bids after a rebid round.
- R-10 The service shall open rebid rounds for baskets in the pending rebid queue and increment the session rebid round counter.
- R-11 The service shall mark the session as **BIDDING_COMPLETED** when all baskets are finalized as **SOLD** or **FINAL_UNSOLD**.
- R-12 The service shall publish domain events through a transactional outbox mechanism to support reliable Kafka delivery.
- R-13 The service shall provide server-rendered dashboard views and snapshot endpoints for live operator monitoring.

4. API / Interface Design

REST / HTTP APIs

Method	Endpoint	Purpose
GET	/	Render the home dashboard listing auction sessions.
GET	/api/home/snapshot	Return a lightweight snapshot token for home page polling.

GET	<code>/auction/:sessionId</code>	Render the auction dashboard for a specific session.
GET	<code>/api/auction/:sessionId/snapshot</code>	Return a lightweight snapshot token for auction page polling.
POST	<code>/auction/:sessionId/start</code>	Start a READY auction session and move it to LIVE .
POST	<code>/auction/:sessionId/baskets/:basketId/open</code>	Open a listed or rebid-ready basket for bidding.
POST	<code>/auction/:sessionId/baskets/:basketId/bid</code>	Place a bid for an open basket.
POST	<code>/auction/:sessionId/baskets/:basketId/close</code>	Close the current basket as sold or unsold and possibly finalize the session.
POST	<code>/auction/:sessionId/rebid/open</code>	Open a rebid round for pending unsold baskets.

No authentication or authorization mechanism is visible in the repository.

Event / Message Interfaces

Interface	Purpose
<code>user.buyer.registered</code>	Carry buyer registration data into the local buyer projection.
<code>catalog.basket.created</code>	Carry basket metadata into the local basket projection.
<code>catalog.published</code>	Trigger creation or update of a local auction session and session baskets.
<code>auction.session.started</code>	Notify downstream consumers that a session has started.
<code>auction.basket.opened</code>	Notify downstream consumers that a basket is open for bidding.
<code>bid.placed</code>	Notify downstream consumers that a valid bid has been accepted.
<code>bid.basket.sold</code>	Notify downstream consumers that a basket was sold to the winning bidder.
<code>bid.basket.unsold</code>	Notify downstream consumers that a basket closed without bids.
<code>bid.rebid.round.opened</code>	Notify downstream consumers that a rebid round has started.

`bid.all.baskets.finalized` Notify downstream consumers that the session bidding phase has fully completed.

Published Kafka messages use JSON payloads, `sessionId` as the message key, and headers including `content-type: application/json` and `eventType`.

5. Data Model / Message Model

The service uses **PostgreSQL** as its private database.

Database Entities / Models

Entity	Main Fields
<code>buyers_projection</code>	<code>buyer_id</code> , <code>name</code> , <code>email</code> , <code>phone</code> , <code>address</code> , <code>occurred_at</code>
<code>catalog_baskets_projection</code>	<code>basket_id</code> , <code>species</code> , <code>quantity</code> , <code>unit</code> , <code>quality</code> , <code>base_price</code> , <code>boat_name</code> , <code>occurred_at</code>
<code>auction_sessions</code>	<code>session_id</code> , <code>title</code> , <code>status</code> , <code>start_time</code> , <code>rebid_round</code> , <code>created_at</code> , <code>updated_at</code>
<code>auction_baskets</code>	<code>basket_id</code> , <code>session_id</code> , <code>basket_no</code> , <code>description</code> , <code>base_price</code> , <code>details_ready</code> , <code>status</code> , <code>highest_bid</code> , <code>opened_at</code> , <code>closed_at</code>
<code>bids</code>	<code>id</code> , <code>session_id</code> , <code>basket_id</code> , <code>bidder_id</code> , <code>bidder_name</code> , <code>amount</code> , <code>placed_at</code>
<code>rebid_queue</code>	<code>id</code> , <code>session_id</code> , <code>basket_id</code> , <code>reason</code> , <code>status</code> , <code>queued_at</code>
<code>event_store</code>	<code>id</code> , <code>event_id</code> , <code>aggregate_type</code> , <code>aggregate_id</code> , <code>event_type</code> , <code>payload</code> , <code>topic</code> , <code>key</code> , <code>occurred_at</code> , <code>published_at</code> , <code>publish_error</code>

Event / Message Models

Event Name	Main Fields
<code>user.buyer.registered</code>	<code>buyerId</code> or <code>id</code> , <code>name</code> , <code>email</code> , <code>phone</code> , <code>address</code> , <code>occurredAt</code>
<code>catalog.basket.created</code>	<code>basketId</code> or <code>id</code> , <code>species</code> , <code>quantity</code> , <code>unit</code> , <code>quality</code> , <code>basePrice</code> , <code>boatName</code> , <code>occurredAt</code>
<code>catalog.published</code>	<code>sessionId</code> or <code>id</code> , <code>title</code> , <code>startTime</code> , <code>basketIds</code>

<code>auction.session.started</code>	<code>eventId, occurredAt, sessionId, startTime, totalBaskets</code>
<code>auction.basket.opened</code>	<code>eventId, occurredAt, sessionId, basketId, basePrice</code>
<code>bid.placed</code>	<code>eventId, occurredAt, sessionId, basketId, buyerId, bidId, bidAmount</code>
<code>bid.basket.sold</code>	<code>eventId, occurredAt, sessionId, basketId, buyerId, winningBidId, salePrice</code>
<code>bid.basket.unsold</code>	<code>eventId, occurredAt, sessionId, basketId, reason</code>
<code>bid.rebid.round.opened</code>	<code>eventId, occurredAt, sessionId, roundNumber, basketIds</code>
<code>bid.all.baskets.finalized</code>	<code>eventId, occurredAt, sessionId, totalBaskets, soldBasketCount, unsoldBasketCount</code>

The upstream event field mappings are inferred from the consumer handlers because the repository does not include formal schema contracts.

6. Events and Communication

Event	Produced / Consumed	Related Service	Purpose
<code>user.buyer.registered</code>	Consumed	Buyer/User service inferred from topic name and README	Populate or update local buyer projection data.
<code>catalog.basket.created</code>	Consumed	Catalog service inferred from topic name and README	Populate or update local basket projection data.
<code>catalog.published</code>	Consumed	Catalog service inferred from topic name and README	Create or refresh local auction session and session basket records.
<code>auction.session.started</code>	Produced	Downstream auction observers or post-auction service inferred from README	Announce that a session has entered the live bidding phase.
<code>auction.basket.opened</code>	Produced	Downstream auction observers or bidder-facing components inferred	Announce that a basket is open for bidding.

<code>bid.placed</code>	Produced	Downstream inferred	observers	Announce that a valid bid has been accepted.
<code>bid.basket.sold</code>	Produced	Post-auction or fulfillment service inferred from README		Announce the winning buyer and sale price for a basket.
<code>bid.basket.unsold</code>	Produced	Post-auction or fulfillment service inferred from README		Announce that a basket closed without bids.
<code>bid.rebid.round.opened</code>	Produced	Downstream inferred	observers	Announce that a rebid round has begun for queued baskets.
<code>bid.all.baskets.finalized</code>	Produced	Post-auction or fulfillment service inferred from README		Announce completion of the bidding phase for the entire session.

The service uses **Kafka** via `kafkajs` as its message broker. It subscribes to three upstream topics and publishes seven downstream topics defined centrally in `src/kafka/topics.js`. Business operations write outbound events to the `event_store` table inside the same database transaction as state changes, then attempt immediate publish after commit. Any unpublished records are retried by a startup flush and a periodic outbox loop. This design makes event flow database-backed and asynchronous while keeping service state local to PostgreSQL.

7. Implementation Summary and Team Contribution

The service is implemented in **Node.js** with **Express**, **EJS**, **Socket.IO**, **KafkaJS**, and **pg**. It uses **PostgreSQL** as its private database and **Kafka** as its inter-service communication mechanism. Main implemented features include buyer and basket projection building, session creation from catalog publication, operator-controlled auction lifecycle management, bid validation and persistence, rebid support, transactional outbox publishing, and a live dashboard with polling and WebSocket refresh behavior.

The repository structure is compact and organized around `src/app.js`, `controllers`, `routes`, `domain`, `db`, `kafka`, `views`, `public`, and `sockets`. The README is present and contains setup and running instructions. The service appears runnable based on `npm` scripts (`start`, `dev`, `db:init`), `.env.example`, database initialization code, and the application bootstrap sequence. The current implementation is functionally substantial, but the repository shows **no automated tests** beyond a placeholder `test` script.

Team Member	Contribution
-------------	--------------

Ufuk Cem Delice	Contributed equally to the design and implementation of the Bidding Service. Responsibilities included service architecture, auction workflow logic, PostgreSQL data handling, Kafka event integration, transactional outbox mechanism, and dashboard-related functionality.
Egehan Vardar	Contributed equally to the design and implementation of the Bidding Service. Responsibilities included service architecture, auction workflow logic, PostgreSQL data handling, Kafka event integration, transactional outbox mechanism, and dashboard-related functionality.
Mustafa Şengül	No active implementation contribution is documented in the final repository state.